OS WEEK 5

1. **praksem1.c**

   ps axj | grep praksem1 -> gunakan ini untuk melihat status praksem1
   Jaga-jaga compilanya pakai gcc -o praksem1 praksem1.c -pthread
   karena semaphore termasuk pakai POSIX thread library

   - Membuat sebuah semaphore dengan nilai awal 1
   - Melakukan perulangan sebanyak 10 kali
   - Mengambil nilai semaphore saat ini menggunakan sem_getvalue()
   - Menampilkan nilai semaphore ke layar
   - Melakukan operasi sem_wait() untuk mengurangi nilai semaphore
   - Jika semaphore bernilai 0, proses akan terblokir (menunggu) karena tidak ada proses lain yang menaikkan semaphore dengan sem_post()

```c
#include <semaphore.h>   // Library untuk semaphore (POSIX semaphore)
#include <stdio.h>       // Library untuk printf() dan perror()

#define INITVALUE 1      // Nilai awal semaphore = 1
#define MAXCOUNT 10      // Perulangan sebanyak 10 kali

int main()
{
    sem_t semid;         // Deklarasi variabel semaphore
    int i, sem_value;    // i untuk counter loop, sem_value untuk menyimpan nilai semaphore

    // Inisialisasi semaphore
    // Parameter:
    // &semid   -> alamat semaphore
    // 0        -> semaphore hanya digunakan dalam proses ini (bukan shared antar proses)
    // INITVALUE -> nilai awal semaphore = 1
    if (sem_init(&semid, 0, INITVALUE) == -1) {
        perror("Failed to initialize semaphore"); // Jika gagal, tampilkan error
        return 1; // Keluar dari program
    }

    // Perulangan sebanyak MAXCOUNT (10 kali)
    for(i = 0; i < MAXCOUNT; i++){

        // Mengambil nilai semaphore saat ini dan simpan ke sem_value
        if (sem_getvalue(&semid, &sem_value) == -1)
            perror("Failed to get semaphore value\n");

        // Cetak nilai semaphore sebelum dilakukan sem_wait
        printf("Semaphore_value = %d \n", sem_value);


        // Mengurangi nilai semaphore sebesar 1 (down operation)
        // Jika nilai semaphore = 0, proses akan BLOCK (menunggu)
        if (sem_wait(&semid) == -1)   // down semaphore, atomically
            perror("Failed to decrement semaphore\n");
    }

    return 0; // Program selesai
}
```

2. **praksem1_varian.c**
   Program modifikasi dari praksem1 dengan menambahkan sem_post(). Dua cara untuk menambahkan sem_post() itu bisa pakai if

   if (sem_post(&semid) == -1)
   perror("Failed to increment semaphore");

biar ada validasinya.
Atau cukup sederhana saja. sem_post(&semid);

- Membuat sebuah semaphore dengan nilai awal 1
- Melakukan perulangan sebanyak 10 kali
- Mengambil nilai semaphore menggunakan sem_getvalue
- Menampilkan nilai semaphore ke layar
- Mengurangi nilai semaphore menggunakan sem_wait()
- Menambahkan kembali nilai semaphore menggunakan sem_post()
- Menghapus semaphore dari memori setelah program selesai

```c
#include <semaphore.h>   // Library untuk semaphore POSIX
#include <stdio.h>       // Library untuk printf() dan perror()

#define INITVALUE 1      // Nilai awal semaphore
#define MAXCOUNT 10      // Jumlah perulangan

int main()
{
    sem_t semid;         // Variabel semaphore
    int i, sem_value;    // i untuk loop, sem_value untuk menyimpan nilai semaphore

    // Inisialisasi semaphore
    // 0 = hanya digunakan dalam proses ini (bukan antar proses)
    // INITVALUE = nilai awal semaphore
    if (sem_init(&semid, 0, INITVALUE) == -1) {
        perror("Failed to initialize semaphore");
        return 1;
    }

    // Loop sebanyak 10 kali
    for(i = 0; i < MAXCOUNT; i++){

        // Ambil nilai semaphore saat ini
        if (sem_getvalue(&semid, &sem_value) == -1)
            perror("Failed to get semaphore value");

        // Cetak nilai semaphore sebelum dilakukan sem_wait
        printf("Semaphore value = %d \n", sem_value);

        // Operasi DOWN (mengurangi nilai semaphore)
        if (sem_wait(&semid) == -1)
            perror("Failed to decrement semaphore");

        // Operasi UP (menambah nilai semaphore kembali)
        /*if (sem_post(&semid) == -1)
            perror("Failed to increment semaphore");*/
            sem_post(&semid);
    }

    // Hapus semaphore dari memori
    sem_destroy(&semid);

    return 0;
}
```

3. **praksem2.c**
Jangan lupa pakai gcc -o praksem2 praksem2.c -pthread juga

- Membuat dua semaphore untuk mengatur komunikasi antara dua thread
- Membuat dua thread yaitu task1 dan task2
- Thread 1 melakukan increment variabel global
- Setiap kelipatan 5, thread 1 memberi sinyal ke thread 2
- Thread 2 akan menampilkan nilai g1
- Setelah thread 2 selesai, thread 1 akan melanjutkan proses
- Program berhenti ketika nilai g1 mencapai 500

```c
#include <stdlib.h>     // Library fungsi umum
#include <unistd.h>     // Library standar Unix
#include <pthread.h>    // Library untuk thread
#include <semaphore.h>  // Library semaphore POSIX
#include <stdio.h>      // Library input output

#define NLOOP 500       // Loop maksimal 500 kali
int g1 = 0;             // Variabel global yang akan dimodifikasi thread

sem_t semid1, semid2;   // Dua semaphore untuk sinkronisasi dua thread


void task1(void *arg)
{
    int i;

    for(i = 0; i < NLOOP; i++) {
        g1++;   // Increment variabel global

        printf("Task1 count: %d g1:%d\n", i+1, g1);

        // Jika g1 kelipatan 5
        if(g1 % 5 == 0) {
            sem_post(&semid2);   // Bangunkan thread 2
            sem_wait(&semid1);   // Tunggu sampai thread 2 selesai
        }
    }
}


void task2(void *arg)
{
    while(1) {

        sem_wait(&semid2);   // Tunggu sinyal dari thread 1

        if (g1 <= NLOOP)
            printf(">>>>>>>>>>>>>>> task 2 g1: %d\n", g1);

        sem_post(&semid1);   // Izinkan thread 1 lanjut

        if(g1 == NLOOP)
            break;          // Stop jika sudah 500
    }
}
```

```c
int main()
{
    pthread_t thr1, thr2;   // Dua thread

    // Inisialisasi semaphore
    sem_init(&semid1, 0, 0);
    sem_init(&semid2, 0, 0);

    // Buat thread
    pthread_create(&thr1, NULL, (void *)task1, NULL);
    pthread_create(&thr2, NULL, (void *)task2, NULL);

    // Tunggu thread selesai
    pthread_join(thr2, NULL);
    pthread_join(thr1, NULL);

    return 0;
}
```

4. **praksem3.c**

- Buat key shared memory untuk nyimpan integer yang bisa dibagi-bagi
- Buat key 2 untuk nyimpan semaphore
- Buat shared memory 1 pakai shmget() untuk integernya
- Buat shared memory 2 pakai shmget() untuk semaphore nya
- Menghubungkan (attach) shared memory integer ke pointer ptr menggunakan shmat()
- Inisialisasi nilai shared integer jadi 0
- Menghubungkan shared memory semaphore ke pointer mutex
- Menginisialisasi semaphore dengan nilai awal 1 menggunakan sem_init() sehingga berfungsi sebagai mutex (binary semaphore).
- Program membuat proses baru menggunakan fork() sehingga ada parent process dan child process.
- Child process melakukan perulangan sebanyak 500000 kali.
- Pada setiap perulangan, child process menambah nilai shared variable (*ptr).
- Child process kemudian menampilkan PID child, PID parent, dan nilai shared variable.
- Parent process juga melakukan perulangan sebanyak 500000 kali.
- Pada setiap perulangan, parent process juga menambah nilai shared variable (*ptr).
- Parent process kemudian menampilkan PID parent, PID parent-nya, dan nilai shared variable.
- Parent process menggunakan wait() untuk menunggu child process selesai.
- Setelah selesai, program melepaskan shared memory menggunakan shmdt().
- Program kemudian menghapus shared memory dari sistem menggunakan shmctl().
- Program selesai dan keluar menggunakan exit(0).

Jadi, shared memory digunakan untuk berbagi data yaitu integernya. Menggunakan semaphore sebagai mutex untuk sinkronisasi. Parent dan child process sama-sama menambah nilai shared variable secara bersamaan.

Terjadi race condition di program ini

```c
#include <stdio.h>      // printf
#include <stdlib.h>     // exit
#include <string.h>
#include <unistd.h>     // fork, getpid
#include <sys/types.h>
#include <sys/wait.h>   // wait
#include <sys/ipc.h>    // IPC
#include <sys/shm.h>    // shared memory
#include <semaphore.h>  // semaphore

#define key  ((key_t) 789)   // key shared memory
#define key2 ((key_t) 7891)  // key semaphore

int main(int argc, char **argv)
{
    int i, nloop=500000, zero=0, value, *ptr;
    int shmid, semid, mode, shm_size, sem_size, status;
    sem_t *mutex;  // pointer ke semaphore di shared memory

    shm_size = sizeof(int);
    shmid = shmget(key, shm_size, 0644 | IPC_CREAT);
    // Membuat shared memory untuk 1 integer

    sem_size = sizeof(sem_t);
    semid = shmget(key2, sem_size, 0644 | IPC_CREAT);
    // Membuat shared memory untuk semaphore

    ptr = (int *)shmat(shmid, (void *)0, 0);
    // Attach shared memory integer

    (*ptr) = 0;
    // Inisialisasi shared integer = 0

    mutex = (sem_t *)shmat(semid, (void *)0, 0);
    // Attach shared memory semaphore

    sem_init(mutex, 1, 1);
    // Inisialisasi semaphore
    // pshared = 1 → digunakan antar proses
    // nilai awal = 1 (binary semaphore / mutex)

    if (fork() == 0) {   // CHILD PROCESS
```

```c
        for (i = 0; i < nloop; i++) {

            value = ++(*ptr);
            // increment shared variable

            printf("child process %d %d: %d\n", getpid(), getppid(), value);
            fflush(NULL);
        }
        exit(0);
    }

    // PARENT PROCESS
    for (i = 0; i < nloop; i++) {

        value = ++(*ptr);
        // increment shared variable

        printf("parent process %d %d: %d\n", getpid(), getppid(), value);
        fflush(NULL);
    }

    wait(&status);  // tunggu child selesai

    shmdt(ptr);
    shmdt(mutex);

    shmctl(shmid, IPC_RMID, NULL);
    shmctl(semid, IPC_RMID, NULL);
    // Hapus shared memory

    exit(0);
}
```

5. praksem3_varian.c

- Program menggunakan shared memory untuk berbagi variabel antar proses.
- Program menggunakan semaphore sebagai mutex untuk melindungi critical section.
- Parent dan child process tidak dapat mengakses shared variable secara bersamaan, sehingga race condition dapat dicegah.

Karena menambah sem_wait() untuk mengunci semaphore, kita tambah, baru pakai dibuka pagi pakai sem_post()

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <semaphore.h>

#define key  ((key_t) 789)
#define key2 ((key_t) 7891)

int main(int argc, char **argv)
{
    int i, nloop = 500000, value, *ptr;
    int shmid, semid, status;
    sem_t *mutex;

    // ========================
    // Membuat shared memory untuk integer
    // ========================
    shmid = shmget(key, sizeof(int), 0644 | IPC_CREAT);
    ptr = (int *) shmat(shmid, NULL, 0);
    *ptr = 0;   // Inisialisasi shared variable

    // ========================
    // Membuat shared memory untuk semaphore
    // ========================
    semid = shmget(key2, sizeof(sem_t), 0644 | IPC_CREAT);
    mutex = (sem_t *) shmat(semid, NULL, 0);

    // Inisialisasi semaphore
    // pshared = 1 → antar proses
    // value = 1 → binary semaphore (mutex)
    sem_init(mutex, 1, 1);

    // ========================
    // Membuat child process
    // ========================
    if (fork() == 0) {   // CHILD

        for (i = 0; i < nloop; i++) {

            // ===== CRITICAL SECTION START =====
            sem_wait(mutex);       // Lock

            value = ++(*ptr);      // Increment shared variable
```

```c
            sem_post(mutex);       // Unlock
            // ===== CRITICAL SECTION END =====

            printf("Child %d value: %d\n", getpid(), value);
        }

        exit(0);
    }

    // ========================
    // PARENT PROCESS
    // ========================
    for (i = 0; i < nloop; i++) {

        // ===== CRITICAL SECTION START =====
        sem_wait(mutex);           // Lock

        value = ++(*ptr);          // Increment shared variable

        sem_post(mutex);           // Unlock
        // ===== CRITICAL SECTION END =====

        printf("Parent %d value: %d\n", getpid(), value);
    }

    wait(&status);   // Tunggu child selesai

    // ========================
    // Cleanup
    // ========================
    shmdt(ptr);
    shmdt(mutex);
    shmctl(shmid, IPC_RMID, NULL);
    shmctl(semid, IPC_RMID, NULL);

    return 0;
}
```

6. client.c

- Program ini menggunakan shared memory untuk berbagi data antar proses.
- Menggunakan POSIX named semaphore untuk sinkronisasi saat membaca data.
- Program menunggu data dari proses lain, kemudian membaca dan mencetak karakter dari shared memory hingga selesai.

```c
#include <unistd.h>     // For sleep(), fork(), etc.
#include <stdlib.h>     // For exit()
#include <sys/types.h>  // For system data types
#include <sys/ipc.h>    // For IPC (Inter-Process Communication)
#include <sys/shm.h>    // For shared memory functions
#include <stdio.h>      // For printf(), putchar()
#include <semaphore.h>  // For POSIX semaphores
#include <sys/stat.h>   // For mode constants
#include <fcntl.h>      // For O_CREAT flag

#define SHMSZ 27        // Define shared memory size (27 bytes)

char SEM_NAME[] = "mysem";   // Name of the POSIX named semaphore

int main()
{
    char ch;                // Character variable (not used here)
    int shmid;              // Shared memory ID
    key_t key;              // Key for shared memory
    char *shm, *s;          // Pointers to shared memory
    sem_t *mutex;           // Semaphore pointer

    // Name the shared memory segment
    key = 1000;             // Assign key value for shared memory

    // Create & initialize semaphore
    // O_CREAT: create if it does not exist
    // 0644: permission (owner read/write, others read)
    // 1: initial semaphore value (unlocked)
    mutex = sem_open(SEM_NAME, O_CREAT, 0644, 1);

    // Create shared memory segment
    // key: unique identifier
    // SHMSZ: size
    // IPC_CREAT | 0666: create if not exists, read/write permission for all
    shmid = shmget(key, SHMSZ, IPC_CREAT | 0666);

    // Attach shared memory segment to process address space
    shm = shmat(shmid, NULL, 0);

    // Wait until first character in shared memory becomes 'A'
    // This acts like a simple synchronization mechanism
    while (*shm != 'A') {
        sleep(1);           // Wait 1 second before checking again
    }

```

```c

    // Read characters from shared memory until null character '\0'
    for (s = shm; *s != '\0'; s++) {

        sem_wait(mutex);     // Lock semaphore (enter critical section)
        putchar(*s);         // Print current character
        sem_post(mutex);     // Unlock semaphore (exit critical section)
    }

    printf("\n");            // Print newline after output

    // Write '*' to shared memory as a signal that reading is done
    *shm = '*';

    sem_close(mutex);        // Close semaphore
    sem_unlink(SEM_NAME);    // Remove named semaphore

    // Remove shared memory segment
    shmctl(shmid, IPC_RMID, 0);

    exit(0);                 // Terminate program successfully
}
```

7. server.c

- Program ini bertindak sebagai server/writer.
- Program menulis huruf A sampai Z ke shared memory.
- Menggunakan semaphore untuk mengatur akses ke shared memory (critical section).

- Program kemudian menunggu client membaca data sebelum shared memory dihapus.

```c
#include <unistd.h>     // fungsi UNIX seperti sleep()
#include <stdlib.h>     // exit()
#include <sys/types.h>  // tipe data sistem
#include <sys/ipc.h>    // IPC (Inter Process Communication)
#include <sys/shm.h>    // shared memory
#include <stdio.h>      // printf()
#include <semaphore.h>  // POSIX semaphore
#include <sys/stat.h>   // permission constants
#include <fcntl.h>      // O_CREAT untuk sem_open()

#define SHMSZ 27        // ukuran shared memory (27 karakter)
char SEM_NAME[] = "mysem";   // nama semaphore (named semaphore)

int main()
{
    char ch;            // variabel untuk menyimpan karakter
    int shmid;          // ID shared memory
    key_t key;          // key untuk shared memory
    char *shm, *s;      // pointer ke shared memory
    sem_t *mutex;       // pointer ke semaphore

    // ================================
    // Membuat shared memory segment
    // ================================
    key = 1000;   // key unik untuk shared memory

    // ================================
    // Membuat & menginisialisasi semaphore
    // ================================
    // sem_open digunakan untuk named semaphore
    // O_CREAT -> buat jika belum ada
    // 0644 -> permission
    // 1 -> nilai awal semaphore (binary semaphore)
    mutex = sem_open(SEM_NAME, O_CREAT, 0644, 1);

    // ================================
    // Membuat shared memory
    // ================================
    shmid = shmget(key, SHMSZ, IPC_CREAT | 0666);
    // key        -> identifikasi shared memory
    // SHMSZ      -> ukuran memori
    // IPC_CREAT  -> buat jika belum ada
    // 0666       -> permission read/write

    // ================================
    // Attach shared memory ke proses
    // ================================
    shm = shmat(shmid, NULL, 0);
```

```c
    // Pointer s untuk mengisi shared memory
    s = shm;

    // ================================
    // Menulis huruf A sampai Z
    // ================================
    for(ch = 'A'; ch <= 'Z'; ch++) {

        sem_wait(mutex);    // LOCK → masuk critical section

        *s++ = ch;          // tulis huruf ke shared memory
                             // pointer s maju satu posisi

        sem_post(mutex);    // UNLOCK → keluar critical section
    }

    // ================================
    // Menunggu client selesai membaca
    // ================================
    // server menunggu sampai karakter pertama berubah jadi '*'
    // biasanya client akan mengubahnya sebagai tanda selesai
    while (*shm != '*') {
        sleep(1);   // tunggu 1 detik
    }

    // ================================
    // Cleanup
    // ================================
    sem_close(mutex);      // tutup semaphore
    sem_unlink(SEM_NAME);  // hapus named semaphore
    shmctl(shmid, IPC_RMID, 0); // hapus shared memory

    exit(0);
}
```

8. rc_week3.c

- Program membuat dua thread yang menambah nilai variabel yang sama (cnt).
- Mutex digunakan untuk mencegah race condition saat kedua thread mengakses variabel tersebut.
- Dengan mutex, nilai akhir cnt akan benar yaitu 20.000.000.

```c
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <pthread.h>

#define NITERS 10000000

void *count(void *arg);

// Shared counter
unsigned int cnt = 0;

// Global mutex
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

int main()
{
    pthread_t tid1, tid2;

    // Create two threads
    pthread_create(&tid1, NULL, count, NULL);
    pthread_create(&tid2, NULL, count, NULL);

    // Wait for both threads to finish
    pthread_join(tid1, NULL);
    pthread_join(tid2, NULL);

    // Check result
    if (cnt != (unsigned) NITERS * 2)
        printf("Wrong... cnt = %u\n", cnt);
    else
        printf("OK...\tcnt = %u\n", cnt);

    return 0;
}
```

```c

void *count(void *arg)
{
    int i;

    for (i = 0; i < NITERS; i++) {

        // ===== Critical Region START =====
        pthread_mutex_lock(&mutex);

        cnt++;   // shared variable modification

        pthread_mutex_unlock(&mutex);
        // ===== Critical Region END =====
    }

    return NULL;
}
```